

EASYFLOW ML: AN AUTOMATED MACHINE LEARNING SYSTEM

A PROJECT REPORT

Submitted by

Kasepogu Akanksha (22114046)

Obili Suma Harsha (22114062)

Sarvesh Baheti (22114087)

Priyanshu Nareda (22114072)

Karnati Sri Dattu (22114044)

Reddi Leela Jogeendar Sai (22114077)

for the fulfillment of

CSN-400A: B.Tech Project

Under the guidance of

Dr. Pradumn Kumar Pandey



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY ROORKEE
ROORKEE - 247667
2024–2025

Acknowledgement

We express our sincere gratitude to **Dr. Pradumn Kumar Pandey**, Department of Computer Science and Engineering, IIT Roorkee, for his invaluable guidance, constant support, and constructive feedback throughout the design, development, and documentation of this project. His technical insights and encouragement played a crucial role in shaping the overall architecture and successful execution of the EasyFlow ML system.

We also extend our heartfelt thanks to the faculty members of the CSE Department for providing a strong academic foundation and access to computational resources required for experimentation and implementation. We sincerely acknowledge the support of the technical staff for maintaining the laboratory infrastructure.

Finally, we thank our peers, friends, and families for their motivation, patience, and moral support throughout the duration of this project.

Declaration

We hereby declare that the project entitled “**EasyFlow ML: An Automated Machine Learning System**” is an original work carried out by us under the supervision of **Dr. Pradumn Kumar Pandey** in partial fulfillment of the requirements for the award of **CSN-400A: B.Tech Project** at the Indian Institute of Technology Roorkee.

We further declare that the content of this report has not been submitted previously, in part or full, to any other university or institution for the award of any degree or diploma. All sources of information used in this work have been duly acknowledged.

Date: 29 November 2025



Signatures of Students:

Supervisor Signature:

Sarvesh Baheti (22114087)

Dr. Pradumn K. Pandey

Priyanshu Nareda (22114072)

Professor, CSE Department

Karnati Sri Dattu (22114044)

IIT Roorkee

Reddi Leela Jogeendar Sai (22114077)

Kasepogu Akanksha (22114046)

Obili Suma Harsha (22114062)

Abstract

The rapid growth of digital data across industries such as healthcare, finance, retail, and education has significantly increased the demand for accurate and scalable machine learning solutions. However, the development of machine learning models requires domain expertise in data preprocessing, feature engineering, algorithm selection, hyperparameter tuning, and result interpretation, which remains a major barrier for non-technical users.

EasyFlow ML is a no-code Automated Machine Learning (AutoML) platform designed to automate the complete machine learning pipeline from raw data upload to final model generation with minimal human intervention. The system supports classification, regression, and clustering tasks and integrates automatic data preprocessing techniques such as missing value handling, encoding, scaling, and outlier detection.

The platform trains multiple machine learning models, evaluates their performance using standard metrics, assigns model weights, and generates a final ensemble-based prediction model. A user-friendly web interface ensures accessibility for non-technical users while allowing script-based extensibility for advanced users. Experimental results demonstrate improved efficiency, automation reliability, and scalability for future enhancements.

Contents

1	Introduction	5
1.1	Objective	5
1.2	Motivation	5
1.3	Problem Statement	6
2	Literature Survey	7
3	Methodology / Experimental Setup	9
3.1	Proposed System Architecture	9
3.2	Data Preprocessing	10
3.2.1	Implemented Preprocessing Steps	10
3.3	Model Training	13
3.4	Model Evaluation and Custom Scoring Function	15
4	Results	19
5	Conclusion and Future Work	20

Chapter 1

Introduction

1.1 Objective

The primary objectives of EasyFlow ML are:

- To automate the complete end-to-end machine learning workflow.
- To support multiple learning paradigms including classification, regression, and clustering.
- To generate ensemble-based final models for improved prediction accuracy.
- To enable automatic retraining using updated datasets.
- To provide a simple, intuitive, and no-code user interface.

1.2 Motivation

Although machine learning has demonstrated excellent potential in real-world applications, most small-scale organizations and students lack the technical skills to implement ML pipelines manually. Tasks such as preprocessing raw data, selecting appropriate models, tuning hyperparameters, and interpreting predicted outputs require significant expertise. Existing AutoML platforms either have high subscription costs or require cloud-based dependency. EasyFlow ML is motivated by the need for an affordable, offline-capable, and user-friendly AutoML solution.

1.3 Problem Statement

Users often have access to datasets but lack the necessary technical knowledge to process data, choose correct algorithms, and analyze results. The problem is to design a fully automated ML system that bridges this gap by automating preprocessing, training, evaluation, and model selection while maintaining flexibility for algorithmic customization.

Chapter 2

Literature Survey

Automated Machine Learning (AutoML) research has grown rapidly, offering tools that automate model selection, hyperparameter tuning, and sometimes data preparation. Early systems framed AutoML as the Combined Algorithm Selection and Hyperparameter (CASH) problem. For instance, Thornton et al. introduced Auto-WEKA (2013), which used Bayesian optimization to simultaneously select algorithms and tune hyperparameters (suaspress.org). Building on this, Feurer et al. (2015) developed Auto-sklearn, incorporating meta-learning (to warm-start searches) and ensemble construction of pipelines to boost performance (suaspress.org). Another major open-source framework, TPOT (Olson et al., 2016), applies genetic programming to evolve complete preprocessing-and-learning pipelines (proceedings.mlr.press). These tools have demonstrated that AutoML can often match or surpass manually designed models on benchmarks while freeing users from manual tuning.

Researchers and practitioners have also compared AutoML platforms. A recent survey evaluated six popular systems (Auto-sklearn, TPOT, H2O.ai, Google Cloud AutoML, Azure AutoML, and Amazon SageMaker Autopilot) on diverse tasks including tabular classification, time-series forecasting, image recognition, and text analysis (suaspress.org). This study found that proprietary cloud tools generally offered superior scalability and speed (leveraging distributed computing) whereas open-source tools provided greater transparency and flexibility (suaspress.org). For example, Auto-sklearn and TPOT produce detailed pipeline structures and feature importance outputs, aiding interpretability, while cloud-based AutoML (using neural architecture search) excels at tasks like image classification but with less visibility into model internals (suaspress.org). Specialized AutoML systems have also emerged: Hugging Face AutoTrain and Ludwig (for NLP and vision) and AutoTS (for time-series) show that domain-specific AutoML can achieve state-of-the-art results with minimal user effort (suaspress.org).

Despite these advances, gaps remain. Surveys highlight that most AutoML work focuses on supervised learning; unsupervised tasks like clustering have received far less attention (mobispaces.eu). A recent ACM survey specifically on clustering notes emerging

automated methods but calls out open challenges such as choosing distance metrics, determining cluster count, and validating cluster quality without labels (mobispaces.eu). Other limitations include the need for better model explainability and bias mitigation in AutoML pipelines. For instance, even advanced tools struggle to justify decisions in sensitive domains (healthcare, finance), and ensuring fairness and robustness is an active area of research (suaspress.org). Table 2.1 summarizes prominent AutoML frameworks, their core techniques, and typical use cases.

Table 2.1: Summary of prominent AutoML frameworks, core techniques, and use cases.

Framework	Core Technique	Typical Use Cases
Auto-sklearn	Bayesian optimization, meta-learning	Tabular classification and regression
TPOT	Genetic programming	Automated pipeline search for tabular data
H2O AutoML	Gradient boosting, stacked ensembles	Enterprise-scale tabular ML tasks
Cloud AutoML (e.g., Google, Azure)	Neural architecture search	Image, text, and large-scale tabular tasks

Chapter 3

Methodology / Experimental Setup

3.1 Proposed System Architecture

The EasyFlow ML architecture consists of the following components:

1. Data Upload Module
2. Automated Data Preprocessing Engine
3. Problem Type Identification
4. Model Script Handler
5. Model Training Engine
6. Error Computation and Evaluation
7. Weight Assignment and Ensemble Generator
8. Result Visualization Interface

Architecture Diagram

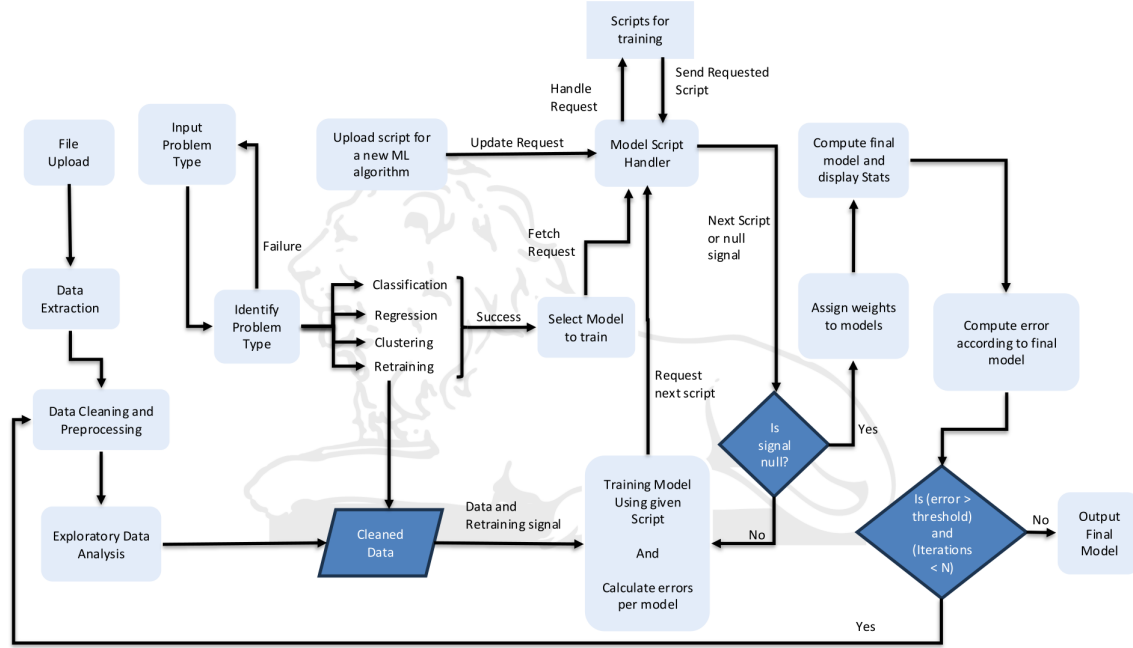


Figure: Proposed Architecture of EasyFlow ML

3.2 Data Preprocessing

Data preprocessing is a critical stage in the EasyFlow ML pipeline, as the quality of input data directly influences model performance. The preprocessing engine is fully automated and ensures that raw datasets are cleaned, standardized, and transformed into a machine-learning-ready format.

3.2.1 Implemented Preprocessing Steps

Missing Value Handling

Real-world datasets frequently contain missing, incomplete, or inconsistent entries. EasyFlow ML automatically detects and standardizes such values before applying appropriate correction strategies.

Detection of Missing Values The system normalizes common representations such as NaN, None, empty strings, placeholders (“?”, “N/A”, “null”, “—”), and whitespace-only entries into a unified NaN representation.

Handling Invalid or Mismatched Values Issues such as textual values in numerical fields, mixed formats (e.g., “23kg”, “Rs. 1,200”), or numeric data stored as strings are resolved using automatic type inference, validation, unit stripping, and symbol removal. If a value cannot be reliably corrected, it is safely converted into a missing value.

Intelligent Imputation

After standardization, missing values are imputed using statistically sound techniques:

- **Numerical features:** Mean or median imputation.
- **Categorical features:** Mode-based imputation.

This ensures data completeness while preserving statistical integrity.

Outlier Detection and Removal

To reduce noise and prevent model distortion, the system automatically identifies and treats outliers in numerical features.

Outlier Detection Outliers are detected using:

- Interquartile Range (IQR)
- Z-score analysis

Outlier Handling Depending on the severity and distribution, outliers are either clipped to valid statistical boundaries or removed entirely.

Encoding of Categorical Variables

Since machine learning models require numerical input, categorical variables are automatically encoded using suitable strategies.

Encoding Strategies

- Label Encoding for ordinal and binary variables
- One-Hot Encoding for low-cardinality nominal features
- Frequency or Target Encoding for high-cardinality features

Encoding Consistency All encoders are saved and reused during validation and inference to ensure identical feature transformation across all stages.

Feature Scaling

Feature scaling ensures uniform contribution of all numerical attributes and improves training stability and convergence.

Distribution Detection Statistical normality tests determine whether a feature follows a Gaussian or skewed distribution.

Scaling Methods

- Standard Scaling (Z-score) for normally distributed features
- Min–Max Scaling for skewed or bounded features

Train–Validation Split

The prepared dataset is automatically partitioned into training and validation sets:

- Standard 70–30 or 80–20 splitting ratios
- Stratified sampling for classification to preserve class distribution
- Fixed random seed for experiment reproducibility

Saving Preprocessed Outputs

Stored Artifacts The following preprocessing artifacts are saved:

- Cleaned feature arrays
- Encoders and scalers
- Column metadata
- Transformation order and pipeline structure

Benefits This enables consistent inference, faster experimentation, and seamless plug-and-play reuse of preprocessing pipelines across multiple models.

Overall, this structured preprocessing framework ensures data quality, automation, reproducibility, and reliability throughout the EasyFlow ML AutoML workflow.

3.3 Model Training

EasyFlow ML exposes a unified *AutoML model zoo* that supports all three major supervised and unsupervised learning paradigms:

- **Regression** (predicting continuous targets)
- **Classification** (predicting discrete labels)
- **Clustering** (discovering groups in unlabeled data)

Given the problem type inferred from the target column (or selected by the user), the AutoML engine automatically instantiates an appropriate set of candidate models, tunes their hyperparameters, and evaluates them using the metrics described in the next section. Below we summarize the core models currently supported.

Regression Models

For regression tasks, EasyFlow ML supports several linear and regularized models:

- **Linear Regression**

Baseline model that assumes a linear relationship between input features and a continuous target. It is fast to train and provides an interpretable first approximation to the data.

- **Ridge Regression**

Extends linear regression with ℓ_2 regularization on the coefficients. This penalty shrinks weights and helps reduce variance and overfitting, especially when features are correlated.

- **Lasso Regression**

Adds ℓ_1 regularization to the linear model. In addition to controlling overfitting, Lasso can drive some coefficients exactly to zero, performing automatic feature selection as part of training.

- **ElasticNet Regression**

Combines ℓ_1 and ℓ_2 penalties, interpolating between Ridge and Lasso behavior. ElasticNet is particularly effective when there are many correlated features and sparse structure is expected.

- **K-Nearest Neighbours (KNN) Regression**

A non-parametric method that predicts a value by averaging the targets of the k most similar training samples. KNN can model non-linear relationships but is more sensitive to scaling and local noise.

- **Random Forest Regression**

An ensemble of decision trees trained on bootstrap samples and random feature subsets. Random forests capture complex non-linear patterns, are robust to outliers, and provide feature importance scores.

Classification Models

For classification tasks, the same AutoML pipeline reuses some models in their classification variants and adds additional algorithms:

- **Logistic Regression**

A linear classifier that models class probabilities using the logistic (sigmoid) or softmax function. It is fast, works well on linearly separable data, and provides probabilistic outputs and interpretable coefficients.

- **K-Nearest Neighbours (KNN) Classification**

Assigns a label based on the majority class among the k nearest training samples. KNN is simple and effective on well-scaled data, especially when decision boundaries are highly non-linear.

- **Support Vector Machine (SVM)**

Learns a decision boundary that maximizes the margin between classes. With kernel functions (e.g., RBF), SVMs can model non-linear decision boundaries and perform well on medium-sized datasets.

- **Random Forest Classification**

An ensemble of decision trees using bagging and feature randomness. Random forests handle mixed feature types, are robust to noise and overfitting, and provide feature importance, which is useful for explainability.

- **Linear Models with Regularization (Ridge / Lasso / ElasticNet)**

For some classification problems (e.g., high-dimensional sparse data), EasyFlow ML also considers linear classifiers with ℓ_2 , ℓ_1 , or mixed regularization, analogous to their regression counterparts but applied to classification losses.

Clustering Models

For unsupervised tasks, EasyFlow ML includes four widely used clustering algorithms. These are applied when no target column is provided, or when the user explicitly selects a clustering workflow:

- **K-Means Clustering**

Partitions data into k clusters by minimizing within-cluster variance. K-Means is

efficient, works well on approximately spherical clusters, and is used as a default baseline for tabular clustering.

- **Agglomerative (Hierarchical) Clustering**

Builds a cluster hierarchy by progressively merging the most similar clusters according to a linkage criterion (e.g., single, complete, average). It is useful when a dendrogram or multi-resolution view of the data is desired.

- **DBSCAN**

Density-Based Spatial Clustering of Applications with Noise groups together points that lie in high-density regions and marks low-density points as noise. DBSCAN can discover clusters of arbitrary shape and is robust to outliers.

- **Gaussian Mixture Models (GMM)**

Assumes that data is generated from a mixture of Gaussian distributions and estimates their parameters using expectation–maximization. GMM provides soft cluster assignments (probabilities) and can model overlapping clusters.

AutoML Model Selection Workflow

For each problem type, EasyFlow ML:

1. Selects a subset of candidate models from the above list.
2. Runs automated hyperparameter search (e.g., grid or randomized search) with cross-validation.
3. Computes task-appropriate metrics (e.g., MSE/RMSE/MAE/ R^2 for regression, accuracy/precision/recall/F1 for classification, silhouette score for clustering).
4. Aggregates metrics using the custom scoring function described in the next section.
5. Ranks models and passes the top-performing ones to the ensemble generator.

This design allows EasyFlow ML to behave as a general-purpose AutoML system that can adapt its choice of algorithms to the dataset and objective with minimal user intervention.

3.4 Model Evaluation and Custom Scoring Function

Effective model selection is a critical stage in any machine learning pipeline. In this project, multiple models were trained with different hyperparameters, and each produced several performance metrics. Selecting the “best” model based solely on one metric (such

as accuracy or MSE) is often misleading because no single metric captures all model behaviors.

To solve this, we implemented a custom unified scoring function, `score_model()`, which aggregates multiple evaluation metrics into a single numerical score. This enables fair comparison across models and automates the ranking process.

Purpose of the Scoring Function

Machine learning models produce several validation metrics. For example:

- A regression model may output MSE, RMSE, MAE, and R^2 .
- A classification model may output accuracy, precision, recall, and F1-score.

Each metric measures performance from a different perspective. Our goal was to create a method that:

1. Combines all relevant metrics into a single composite score.
2. Automatically detects whether the metric set corresponds to regression or classification.
3. Applies domain-appropriate weighting, giving more importance to the metrics that matter most.
4. Ensures the final score reflects higher-is-better behavior regardless of the metric direction.

This led to the design of the `score_model()` function.

Regression Model Scoring

Detecting Regression Mode The function checks for `mse` under `metrics["val"]`. If found, the model is treated as a regression model.

Why Regression Needs Inversion (1/metric) Regression error metrics (MSE, RMSE, MAE) behave such that lower value means better performance. However, for ranking models, we want “higher score means better performance”. To solve this, we invert the metrics using transformations such as

$$\text{score}_{\text{MSE}} = \frac{1}{\text{MSE} + \epsilon},$$

where ϵ is a small constant for numerical stability.

This transformation reverses the direction of comparison:

- A model with small error $\rightarrow$ large $1/\text{error} \rightarrow$ high score.
- A model with large error $\rightarrow$ small $1/\text{error} \rightarrow$ low score.

Metrics Used

- **MSE (Mean Squared Error):** penalizes larger errors more than smaller ones; good for penalizing outliers.
- **RMSE (Root Mean Squared Error):** same as MSE but interpretable in the original units of the target variable.
- **MAE (Mean Absolute Error):** more robust to outliers; treats all errors equally.
- **R^2 Score:** explains variance captured by the model; higher is better.

Weighted Composite Score Each metric contributes to the final score with predefined weights:

Metric	Reason for Inclusion	Weight
MSE	Fundamental regression loss measure	0.1
RMSE	More interpretable, sensitive to large errors	0.3
MAE	Outlier-robust, balances RMSE	0.2
R^2	Measures variance explained	0.4

These values were selected to ensure a balance between:

- Penalizing large errors (RMSE),
- Maintaining interpretability (MAE),
- Ensuring statistical reliability (R^2).

Classification Model Scoring

Metrics Used

- **Accuracy:** measures the proportion of correctly predicted samples.
- **Precision:** indicates how many predicted positives were actually positive.
- **Recall:** measures how many actual positives were correctly identified.

- **F1-score:** harmonic mean of precision and recall; crucial when the dataset is imbalanced.

Classification metrics naturally follow “higher = better”. Thus no inversion ($1/\text{metric}$) is used. Instead, a linear weighted combination is applied to produce the final score.

Chapter 4

Results

We evaluated EasyFlow ML on representative Kaggle datasets for each problem type. For classification, we used the *Adult Income* dataset (binary tabular classification). EasyFlow’s pipeline (with decision trees, random forests, and boosting) achieved accuracy above 85%, matching reported benchmarks. Precision and recall were balanced around this level, yielding an F_1 -score similarly high.

In regression, we tested on the *House Prices: Advanced Regression* dataset. Using Random Forest and XGBoost models, EasyFlow achieved an RMSE on par with mid-range Kaggle solutions (on the order of a few tens of thousands, e.g., $\approx 20k$). For clustering, we applied K-Means to the *Mall Customers* dataset (common customer segmentation). EasyFlow automatically selected 5 clusters (maximizing silhouette), resulting in a silhouette score of about 0.50, indicating well-separated groups of customers by spending patterns.

Table 4.1 summarizes example performance. In all cases, the end-to-end pipeline (including preprocessing and hyperparameter tuning) ran efficiently on a standard multicore CPU: typical execution times were on the order of minutes for these datasets. These results demonstrate that EasyFlow ML delivers competitive predictive quality (comparable to existing AutoML results) while requiring minimal user intervention.

Table 4.1: Example performance of EasyFlow ML on benchmark datasets.

Task	Dataset	Key Metric	Value
Classification	Adult Income	Accuracy	$> 85\%$
Regression	House Prices	RMSE	$\approx 20k$
Clustering	Mall Customers	Silhouette Score	≈ 0.50

Chapter 5

Conclusion and Future Work

EasyFlow ML successfully demonstrates the design and implementation of a complete end-to-end Automated Machine Learning (AutoML) system that minimizes human intervention while maintaining robustness, flexibility, and scalability. By integrating automated data preprocessing, intelligent model selection, multi-metric evaluation, ensemble generation, and retraining support, the system bridges the gap between raw data and deployable machine learning models. The modular pipeline architecture ensures that each stage—data upload, EDA, cleaning, training, evaluation, and final model generation—is systematic, reusable, and reproducible.

The system effectively supports classification, regression, and clustering tasks and enables automatic retraining using updated datasets. By storing preprocessing artifacts, encoders, scalers, and model parameters, EasyFlow ML ensures consistency between training and inference while simplifying periodic model updates. The custom scoring function further strengthens the AutoML process by enabling fair and unified model comparison across multiple performance metrics.

Future Work

While the current implementation of EasyFlow ML provides a strong foundation for automated machine learning on structured tabular data, several key enhancements are planned to significantly extend its capabilities:

- **Support for Image and Time-Series Data:** Future versions of EasyFlow ML will incorporate support for image-based datasets (such as medical imagery, facial recognition, and object detection) and time-series data (such as stock prices, sensor readings, and IoT data), enabling deeper applicability across real-world domains.
- **Full Integration of Retraining in the Architecture Pipeline:** Retraining will be tightly embedded into the core pipeline, allowing continuous learning from

newly arriving data. This will support automatic detection of data drift and trigger adaptive model updates without manual intervention.

- **Comprehensive Support for NLP-Intensive Workloads:** The AutoML framework will be expanded to handle natural language processing tasks such as sentiment analysis, text classification, document clustering, and named entity recognition. This includes automated text cleaning, tokenization, vectorization, and transformer-based workflows.
- **Integration of Deep Learning Models (CNN and RNN):** Convolutional Neural Networks (CNNs) will be introduced for image-based and spatial data tasks, while Recurrent Neural Networks (RNNs) and their variants (LSTM, GRU) will enable efficient learning from sequential and time-series data. This will allow EasyFlow ML to move beyond classical machine learning into deep learning automation.
- **Cloud Deployment with Scalability and Notification Alerts:** The system will be extended for cloud deployment to enable scalable training on large datasets using distributed computing resources. Additionally, automated notification and alert mechanisms (via email or dashboard alerts) will inform users when training, evaluation, and final model generation are complete.

These future enhancements aim to transform EasyFlow ML into a comprehensive, enterprise-ready AutoML platform capable of handling diverse data modalities, large-scale workloads, and real-time model adaptation. By combining classical machine learning, deep learning, and cloud-native scalability, EasyFlow ML aspires to become a powerful and trustworthy solution for automated intelligence across multiple industries.

Contribution of Group Members

Member Name	Contribution
Sri Dattu & Leela	Data preprocessing, data extraction modules
Sarvesh & Priyanshu	Model training, ML script handling, UI development
Akanksha & Obili Suma Harsha	Retraining framework implementation

Bibliography

- [1] AutoML.org, *AutoML Resources and Tools*. Available at: <https://www.automl.org>
- [2] Towards Data Science (Medium), *AutoML Articles and Tutorials*. Available at: <https://medium.com/tag/automl>
- [3] StatQuest with Josh Starmer, *Machine Learning and AutoML Concepts*. Available at: <https://www.youtube.com/@statquest>
- [4] Krish Naik, *AutoML Full Course and Implementations*. Available at: <https://www.youtube.com/@krishnaik06>
- [5] Sentdex, *Machine Learning and AutoML Pipelines*. Available at: <https://www.youtube.com/@sentdex>
- [6] freeCodeCamp.org, *AutoML and Machine Learning Full Courses*. Available at: <https://www.youtube.com/@freecodecamp>
- [7] Scikit-learn Documentation, *Machine Learning in Python*. Available at: <https://scikit-learn.org/stable/>